

[Наименование образовательной организации]

[Факультет или институт]

[Кафедра]

ОТЧЁТ

по лабораторной работе № 2

Кластеризация

по дисциплине

[Наименование дисциплины]

Выполнил: [ФИО студента]

Группа: [Номер группы]

Проверил: [ФИО преподавателя]

Подпись: _____

Дата: _____

[Город] 2026

СОДЕРЖАНИЕ

1 Цель и задачи работы	3
2 Краткие теоретические сведения	4
3 Исходные данные и среда выполнения	5
4 Подробный процесс выполнения	6
5 Подбор параметров	7
6 Метрики	8
7 Экспертная оценка	9
8 Визуализация	10
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13
ПРИЛОЖЕНИЕ А	14

1 Цель и задачи работы

Получить и закрепить навыки предварительной обработки данных и применения методов кластеризации для выделения и содержательной оценки групп заёмщиков.

Для достижения цели поставлены следующие задачи:

- Подготовить общие признаки без `loan_status` и выбрать одинаковые объекты для всех методов.
- Обучить KMeans, DBSCAN и иерархическую кластеризацию, подобрать параметры.
- Оценить разделённость и сопоставить кластеры с реальными классами дефолта.
- Визуализировать разбиения и дендрограмму, исследовать профили и размеры групп.
- Дать экспертную оценку и объяснить ограничения найденного разбиения.

2 Краткие теоретические сведения

Кластеризация - обучение без учителя: объекты объединяются по сходству признаков без использования известных меток. Кластер не обязан совпадать с классом дефолта, а номер кластера не имеет самостоятельного экономического значения.

KMeans минимизирует сумму квадратов евклидовых расстояний объектов до центров их групп. Число групп задаётся заранее; несколько инициализаций уменьшают зависимость от начальных центров. DBSCAN находит плотные области по радиусу ϵ и минимальному числу соседей min_samples . Объекты, не присоединённые к плотным группам, считаются шумом и получают метку -1. Иерархическая агломеративная кластеризация начинается с отдельных объектов и последовательно объединяет группы; правило linkage определяет расстояние между ними. Ward ориентируется на прирост внутригрупповой суммы квадратов, average - на среднее межгрупповое расстояние [3].

Silhouette для объекта определяется как $s = (b - a) / \max(a, b)$, где a - среднее расстояние внутри своей группы, b - минимальное среднее расстояние до другой группы. Большая оценка означает хорошее геометрическое разделение, но не доказывает содержательную полезность кластеров. ARI и AMI сравнивают разбиение с известными классами с поправкой на случайные совпадения. Номера групп при таком сравнении можно переставлять без изменения метрики.

PCA строит ортогональные направления большой дисперсии. Проекция на две компоненты используется для визуализации, но может терять существенную часть информации. Экспертная оценка учитывает размеры групп, средние значения, типичные категории и доли дефолтов. Небольшая группа или шум не означают автоматически ошибку данных.

3 Исходные данные и среда выполнения

Использован Credit Risk Dataset автора Lao Tse с Kaggle, лицензия CC0 [2]. Таблица содержит 32 581 запись и 12 столбцов. В `person_emp_length` отсутствуют 895 значений, в `loan_int_rate` - 3116. После удаления строк с пропусками и 137 дубликатов осталось 28 501 наблюдение. Число удалённых строк с пропусками равно 3943, поскольку пропуски в двух столбцах могут встречаться в одной строке.

Числовые столбцы описывают возраст, доход, стаж, сумму кредита, ставку, отношение кредита к доходу и длительность кредитной истории. Категории описывают жильё, цель кредита, категорию кредита и историю дефолтов. `loan_status` - бинарный исход кредита. Исходный файл находится в `data/raw/credit_risk_dataset.csv` и сохраняется без изменений.

Расчёты выполнены средствами Python 3.12.14 и `scikit-learn` 1.9.1. `pandas` используется для таблиц, `NumPy` - для массивов, `matplotlib` и `seaborn` для графиков. Точные версии зависимостей сохранены в `requirements-lock.txt` соответствующей лабораторной. Аномалии возраста и стажа не удалялись; они учитываются как ограничение интерпретации результатов.

4 Подробный процесс выполнения

Шаг 1. Из общего исходного CSV удалены пропуски и дубликаты. После удаления `loan_status` сформированы числовые и категориальные признаки. Для сопоставимого эксперимента `clean.sample` с `random_state=42` выбирает 4000 одинаковых строк для всех методов; цель не используется при отборе.

Шаг 2. `ColumnTransformer` объединяет `StandardScaler` и `OneHotEncoder`. Получено 26 преобразованных признаков. Масштабирование выполняется по этой исследовательской выборке; отдельного теста нет, поскольку исследуются группы имеющихся объектов, а не прогноз для новых объектов.

Шаг 3. `KMeans` проверяет число групп 2–8, `n_init=10`. `DBSCAN` проверяет 9 значений `eps` и `min_samples=3/5/10/25`. Иерархический метод проверяет число групп 2–8 и `linkage=ward/average`. Все модели работают в полном пространстве признаков.

Шаг 4. Внутренний критерий - `silhouette` на не более чем 1500 объектах с фиксированным `seed`. Для `DBSCAN` шум исключён, требуется `coverage ≥ 0.5` и не менее двух групп. Оценка выбора равна `silhouette` минус доля шума. Это эвристическое правило; оно не гарантирует равномерных или полезных групп.

Шаг 5. После выбора параметров рассчитаны `ARI`, `AMI`, `homogeneity` и `completeness` по `loan_status`. Шум учитывается как отдельная группа -1. Для интерпретации сохранены численности, средние числовые признаки, типичные категории и доли дефолтов.

Шаг 6. `PCA` используется только для двумерного рисунка. Дендрограмма отдельно построена по 150 объектам с выбранным правилом `average`. Файл `view_results.py` открывает метрики, профили и графики. Снимок страницы приведён ниже; полный вычислительный код расположен в приложении А.

Команда просмотра из корня проекта: `.\venv\Scripts\python Лаба2/view_results.py`. Для повторного расчёта к команде добавляется `--rebuild`.

На рисунке 1 представлен результат этапа: снимок локальной страницы с результатами лабораторной.

Лабораторная №2. Кластеризация

После очистки: 28501 строк. Все методы сравниваются на одинаковой случайной выборке из 4000 строк. loan_status исключён из признаков и использован только для оценки.

Метрики

method	clusters	coverage	noise_rows	silhouette	ARI	AMI	
KMeans	2	1.0	0	0.19951888476403526	-0.024042442033367542	0.003166659773166214	0.0
DBSCAN	2	0.9955	18	0.49827936821028523	0.005011329817932747	0.0018480251365370502	0.0
Hierarchical	3	1.0	0	0.5162407801090124	0.011713238842198477	0.0035387437021334508	0.0

Silhouette оценивает разделённость групп; шум DBSCAN исключён из него. Coverage — доля объектов без шума. ARI и AMI сравнивают разбиения независимо от номеров кластеров: близость к нулю означает слабое соответствие реальным классам. В сравнении классов шум -1 учитывается как отдельная группа.

Подобранные параметры

```
{
  "KMeans": {
    "n_clusters": 2
  },
  "DBSCAN": {
    "eps": 3.0,
    "min_samples": 3
  },
  "Hierarchical": {
    "n_clusters": 3,
    "linkage": "average"
  }
}
```

Экспортная оценка

Рисунок 1 - Снимок локальной страницы с результатами лабораторной

5 Подбор параметров

KMeans: k=2–8, n_init=10. DBSCAN: 9 значений eps от 0.8 до 3 и четыре значения min_samples (3, 5, 10, 25). Иерархический метод: k=2–8 и linkage=ward/average. Критерий — silhouette (максимум на 1500 объектах). Для DBSCAN silhouette считается без шума, требуется coverage ≥ 0.5 и не менее двух кластеров; из silhouette вычитается доля шума. Реальные классы не использованы для выбора параметров.

В таблице 1 приведены результаты сравнения моделей.

Таблица 1 - Результаты сравнения моделей

Метод	Выбранные параметры
KMeans	n_clusters=2
DBSCAN	eps=3, min_samples=3
Иерархический	n_clusters=3, linkage=average

6 Метрики

В таблице 2 приведены результаты сравнения моделей.

Таблица 2 - Результаты сравнения моделей

Метод	Кластеры	Шум	Silhouette	ARI	AMI
KMeans	2	0	0.1995	-0.0240	0.0032
DBSCAN	2	18	0.4983	0.0050	0.0018
Иерархический	3	0	0.5162	0.0117	0.0035

Silhouette отражает разделённость групп в пространстве признаков, а не соответствие дефолту. ARI/AMI близки к нулю: найденные группы слабо согласуются с loan_status. Отрицательный ARI KMeans означает результат ниже ожидаемого совпадения случайных разбиений при фиксированных размерах групп. Номера кластеров нельзя напрямую сравнивать с 0/1. ARI и AMI не зависят от перестановки номеров. Шум DBSCAN учтён как отдельная группа в этих метриках.

На рисунке 2 представлен результат этапа: сравнение с реальными классами.

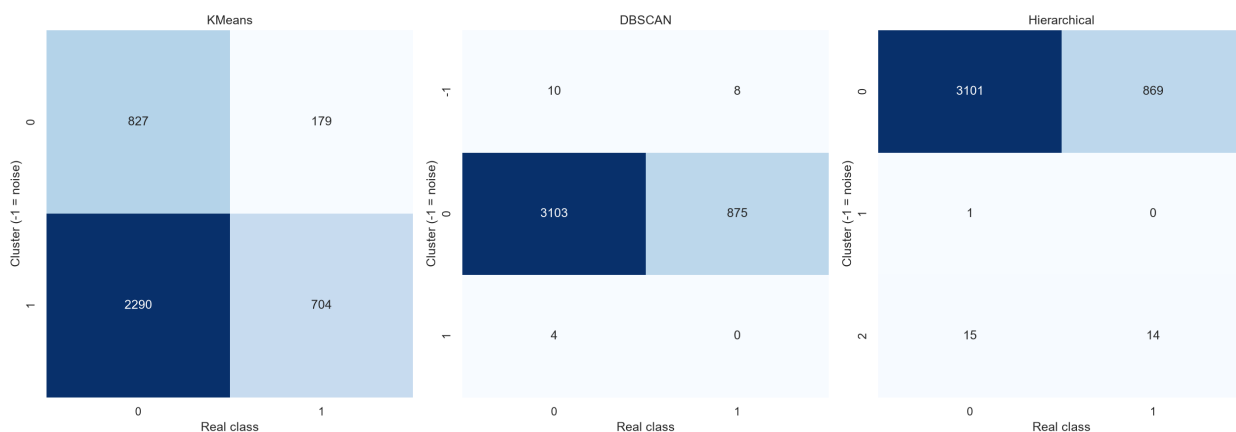


Рисунок 2 - Сравнение с реальными классами

7 Экспертная оценка

KMeans образует группы из 1006 и 2994 объектов. В первой средний возраст 35.1 года, доход 90 779, длина кредитной истории 10.8 года; во второй — 25.1 года, доход 60 470 и история 4.1 года. Доли дефолтов 17.8% и 23.5%. Это интерпретируемое разделение по возрасту и кредитной истории, но оно не является разделением на дефолтных и недефолтных заёмщиков.

DBSCAN: основная группа — 3978 объектов с долей дефолтов 22.0%, малая — 4 объекта без дефолтов, шум — 18 объектов, из них 8 с дефолтом. Малая группа имеет средний возраст 55.3 года и кредитную историю 23 года. Шум отличается высокими средними доходами и возрастом. Из-за малого размера групп оценка их доли дефолтов нестабильна; шум не означает автоматически ошибку данных.

Иерархический метод: основная группа — 3970 объектов, вторая — один объект с возрастом 144 года и доходом 6 млн, третья — 29 объектов со средним возрастом 55.9 года и историей 24.1 года. Доли дефолтов — 21.9%, 0% и 48.3%. Одиночный аномальный объект показывает чувствительность метода к выбросам. Высокий silhouette здесь не достаточен для признания разбиения полезным.

Для содержательного описания массовых сегментов KMeans удобнее в этом запуске; DBSCAN и иерархический метод преимущественно отделяют небольшие необычные группы. Утверждения основаны на профилях групп, а не на внешней экспертизе кредитного специалиста.

8 Визуализация

На рисунке 3 представлен результат этапа: рса и разбиения.

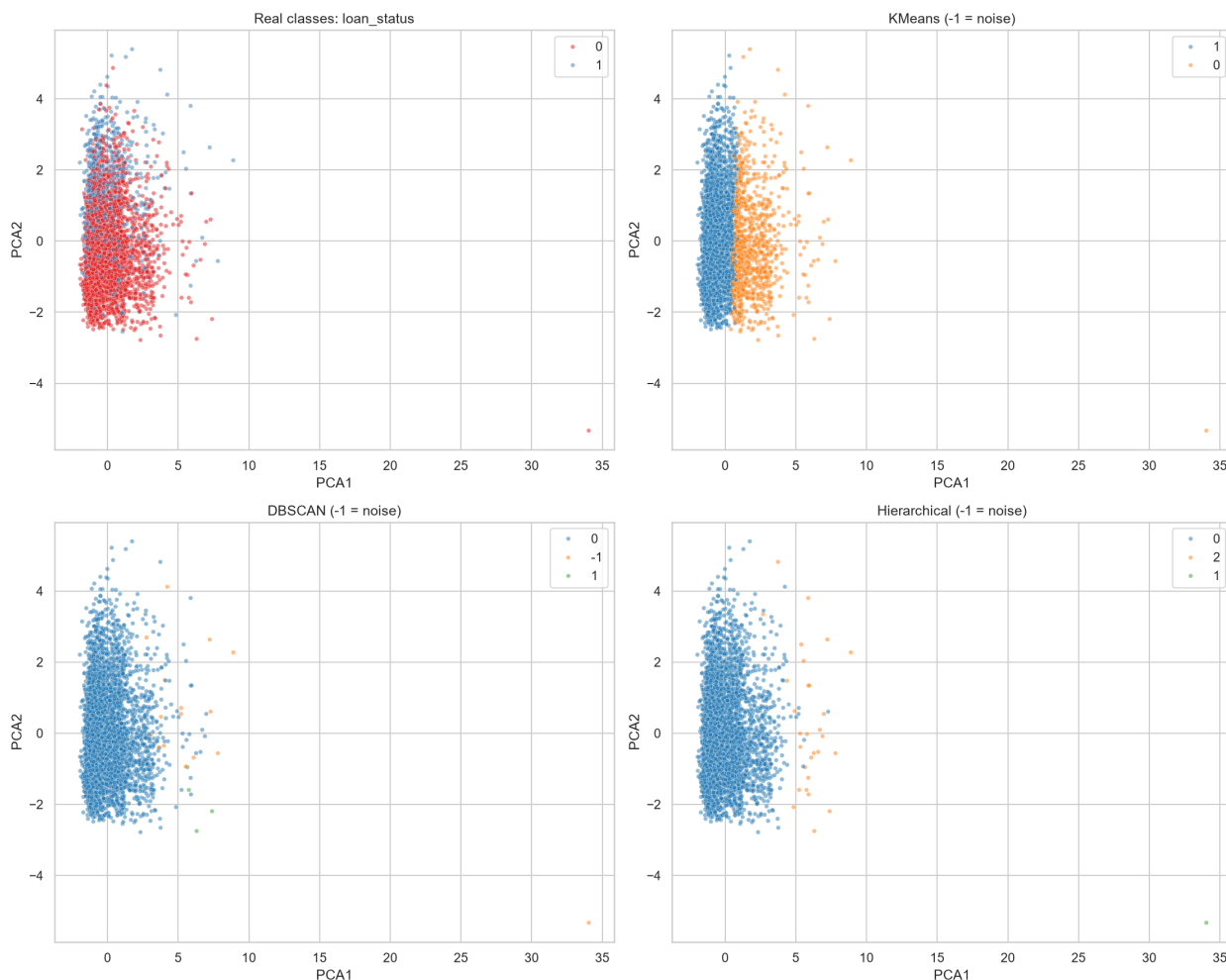


Рисунок 3 - PCA и разбиения

Две компоненты PCA сохраняют 39.97% дисперсии. Модели обучались на всех преобразованных признаках; пересечение групп на 2D-проекции не доказывает их пересечение в полном пространстве. Цвета кластеров разных моделей не означают соответствие групп друг другу.

На рисунке 4 представлен результат этапа: дендрограмма.

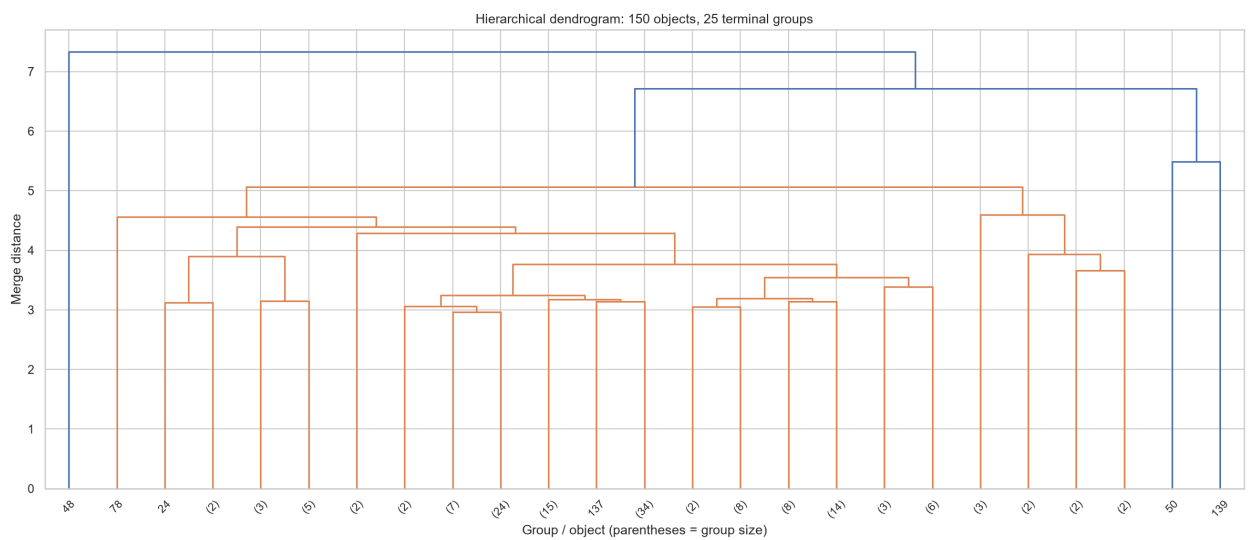


Рисунок 4 - Дендрограмма

Дендрограмма иллюстрирует выбранный способ объединения average на отдельной подвыборке из 150 объектов. Последние 25 групп показаны в сокращённом виде.

ЗАКЛЮЧЕНИЕ

Цель достигнута: подготовлены признаки без целевой метки, обучены и настроены KMeans, DBSCAN и иерархический метод. Выполнены экспертная оценка, сравнение с реальными классами и визуализация. ARI около нуля показывает, что кластеры не воспроизводят дефолт. KMeans даёт интерпретируемые возрастные сегменты, два других метода отделяют малые необычные группы. Отрицательный результат сопоставления классов не означает невыполнение задачи кластеризации. Вывод относится к случайной выборке из 4000 объектов и выбранному представлению.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления. URL: <https://meganorm.ru/Data2/1/4294847/4294847312.htm> (дата обращения: 03.10.2026).
2. Lao Tse. Credit Risk Dataset. Kaggle. URL: <https://www.kaggle.com/datasets/laotse/credit-risk-dataset> (дата обращения: 03.10.2026).
3. Scikit-learn. Руководство пользователя. URL: <https://scikit-learn.org/stable/modules/clustering.html> (дата обращения: 03.10.2026).
4. Исходный код и сохранённые результаты проекта ОМО. URL: <https://github.com/Gemoroyy1/ОМО> (дата обращения: 03.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код clustering.py

Приведён полный исходный текст файла Лаба2/clustering.py. Длинные строки перенесены только для печати; исходный файл в репозитории сохраняет исходные переносы и отступы. Код выполнялся в среде Python проекта.

```
"""Лабораторная №2. Запуск: python Лаба2/clustering.py."""
```

```
import os
from pathlib import Path
import json

ROOT = Path(__file__).resolve().parent
os.environ.setdefault("MPLCONFIGDIR", str(ROOT /
    ".cache/matplotlib"))
os.environ.setdefault("OMP_NUM_THREADS", "2")
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import KMeans, DBSCAN,
    AgglomerativeClustering
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.metrics import (silhouette_score,
    adjusted_rand_score,
                                adjusted_mutual_info_score,
                                homogeneity_score,
                                completeness_score)
from sklearn.preprocessing import OneHotEncoder, StandardScaler

NUMERIC = ["person_age", "person_income", "person_emp_length",
    "loan_amnt",
    "loan_int_rate", "loan_percent_income",
    "cb_person_cred_hist_length"]
TARGET = "loan_status"
OUTPUT = ROOT / "reports"
```

```

def save(name):
    plt.tight_layout()
    plt.savefig(OUTPUT / name, dpi=150, bbox_inches="tight")
    plt.close()

def internal_score(x, labels):
    """Шум исключён из silhouette; coverage учитывает его долю
    отдельно."""
    mask = labels != -1
    count = len(np.unique(labels[mask]))
    coverage = float(mask.mean())
    if count < 2 or count >= mask.sum():
        return count, coverage, None
    # Фиксированная подвыборка ограничивает стоимость попарных
    # расстояний.
    try:
        score = float(silhouette_score(x[mask], labels[mask],
                                         sample_size=min(1500,
int(mask.sum()))), random_state=42))
    except ValueError:
        score = None
    return count, coverage, score

def main():
    OUTPUT.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid")
    source = ROOT.parent / "data/raw/credit_risk_dataset.csv"
    raw = pd.read_csv(source)
    complete = raw.dropna()
    clean = complete.drop_duplicates()
    # Одинаковая случайная выборка для всех методов; цель не
    # участвует в отборе.
    data = clean.sample(n=min(4000, len(clean)),
random_state=42).copy()
    features = data.drop(columns=TARGET)
    categories = [name for name in features if name not in
NUMERIC]
    encoder = ColumnTransformer([
        ("numeric", StandardScaler(), NUMERIC),
        ("category", OneHotEncoder(handle_unknown="ignore",
sparse_output=False), categories),
    ])
    x = encoder.fit_transform(features)
    candidates, selected = [], {}
    for method in ["KMeans", "DBSCAN", "Hierarchical"]:

```

```

print(f"Selecting {method}...", flush=True)
best_value = -np.inf
configs = ([{"n_clusters": k} for k in range(2, 9)] if
method == "KMeans"
            else [{"eps": eps, "min_samples": n}
                    for eps in [0.8, 1.0, 1.2, 1.4, 1.6,
1.8, 2.0, 2.5, 3.0] for n in [3, 5, 10, 25]]
            if method == "DBSCAN" else
            [{"n_clusters": k, "linkage": how} for k in
range(2, 9)
              for how in ["ward", "average"]])
for config in configs:
    model = (KMeans(**config, random_state=42,
n_init=10) if method == "KMeans"
            else DBSCAN(**config, n_jobs=2) if method
== "DBSCAN"
            else AgglomerativeClustering(**config))
    labels = model.fit_predict(x)
    count, coverage, silhouette = internal_score(x,
labels)
    # DBSCAN: не допускаем выбора малого островка с
высоким silhouette.
    valid = silhouette is not None and coverage >= 0.5
    value = silhouette - (1 - coverage) if valid else
None
    candidates.append({"method": method, "parameters":
json.dumps(config),
                      "clusters": count, "coverage":
coverage,
                      "silhouette": silhouette,
"selection_score": value})
    if value is not None and value > best_value:
        best_value = value
        selected[method] = (labels.copy(), config)
    if method not in selected:
        pd.DataFrame(candidates).to_csv(OUTPUT /
"parameter_search.csv", index=False)
        raise RuntimeError(f"No valid {method} partition:
inspect grid and representation")
    print(method, selected[method][1], flush=True)
pd.DataFrame(candidates).to_csv(OUTPUT /
"parameter_search.csv", index=False)
pca = PCA(n_components=2, random_state=42)
points = pca.fit_transform(x)
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
sns.scatterplot(x=points[:, 0], y=points[:, 1],
hue=data[TARGET].to_numpy(),
                palette="Set1", s=14, alpha=0.5, ax=axes[0,

```

```

0])
axes[0, 0].set_title("Real classes: loan_status")
metrics, profiles, comparisons = [], [], []
predictions = pd.DataFrame({"source_row": data.index,
                             "actual": data[TARGET].to_numpy(),
                             "PCA1": points[:, 0], "PCA2":
points[:, 1]})
for ax, (method, (labels, config)) in
zip(list(axes.flat)[1:], selected.items()):
    count, coverage, silhouette = internal_score(x, labels)
    y = data[TARGET].to_numpy()
    metrics.append({"method": method, "clusters": count,
"coverage": coverage,
                    "noise_rows": int((labels == -1).sum()),
    "silhouette": silhouette,
                    "ARI": adjusted_rand_score(y, labels),
                    "AMI": adjusted_mutual_info_score(y,
labels),
                    "homogeneity": homogeneity_score(y,
labels),
                    "completeness": completeness_score(y,
labels)})
    predictions[method] = labels
    sns.scatterplot(x=points[:, 0], y=points[:, 1],
hue=labels.astype(str),
                    palette="tab10", alpha=0.5, s=14, ax=ax)
    ax.set_title(method + " (-1 = noise)")
    grouped = data.assign(cluster=labels).groupby("cluster")
    summary = grouped[numeric].mean()
    summary["rows"] = grouped.size()
    summary["default_rate"] = grouped[TARGET].mean()
    for category in categories:
        summary[category + "_mode"] =
grouped[category].agg(lambda s: s.mode().iloc[0])
    summary.insert(0, "method", method)
    profiles.append(summary.reset_index())
    cross = pd.crosstab(labels, y).reindex(columns=[0, 1],
fill_value=0)
    cross.index.name = "cluster"
    cross.insert(0, "method", method)
    comparisons.append(cross.reset_index())
for ax in axes.flat:
    ax.set_xlabel("PCA1")
    ax.set_ylabel("PCA2")
save("partitions_pca.png")
fig, axes = plt.subplots(1, 3, figsize=(17, 6))
for ax, (method, (labels, _)) in zip(axes,
selected.items()):

```

```

        table = pd.crosstab(labels,
data[TARGET].to_numpy()).reindex(columns=[0, 1],
fill_value=0)
        sns.heatmap(table, annot=True, fmt="d", cmap="Blues",
ax=ax, cbar=False)
        ax.set_title(method)
        ax.set_xlabel("Real class")
        ax.set_ylabel("Cluster (-1 = noise)")
save("classes_comparison.png")
# Отдельная иллюстративная дендрограмма: 150 объектов из той
же выборки.
plt.figure(figsize=(16, 7))
tree = linkage(x[:150],
method=selected["Hierarchical"][1]["linkage"])
dendrogram(tree, truncate_mode="lastp", p=25)
plt.title("Hierarchical dendrogram: 150 objects, 25 terminal
groups")
plt.xlabel("Group / object (parentheses = group size)")
plt.ylabel("Merge distance")
save("dendrogram.png")
pd.DataFrame(metrics).to_csv(OUTPUT / "metrics.csv",
index=False)
pd.concat(profiles, ignore_index=True).to_csv(OUTPUT /
"cluster_profiles.csv", index=False)
pd.concat(comparisons, ignore_index=True).to_csv(OUTPUT /
"class_counts.csv", index=False)
predictions.to_csv(OUTPUT / "predictions.csv", index=False)
details = {"source": "data/raw/credit_risk_dataset.csv",
"original_rows": len(row),
"removed_missing": len(row) - len(complete),
"removed_duplicates": len(complete)-len(clean),
"clean_rows": len(clean), "sample_rows":
len(data), "random_state": 42,
"encoded_features":
list(encoder.get_feature_names_out()),
"pca_explained_variance":
pca.explained_variance_ratio_.tolist(),
"selected_parameters": {method: config for
method, (_, config) in selected.items()}}
(OUTPUT / "results.json").write_text(json.dumps(details,
ensure_ascii=False, indent=2), encoding="utf-8")
print(pd.DataFrame(metrics).to_string(index=False))

```

```

if __name__ == "__main__":
    main()

```